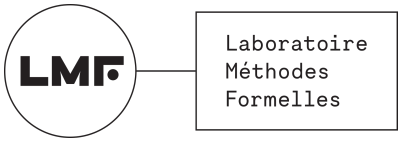

Research project (TER) report - Spring 2026

Distributed framework for #SAT solvers

Thanh Phat Doan

Under the supervision of

Mihaela Sighireanu

Formal Methods Laboratory (LMF)

ENS Paris-Saclay

April 1, 2026

Abstract

Model counting (#SAT) is a fundamental problem in automated reasoning with applications ranging from program verification to probabilistic inference. While significant progress has been made in sequential model counters, many real-world instances remain computationally intractable for single-machine solving. This project presents a distributed framework for model counting that decomposes the search space into independent subproblems and distributes them across multiple workers. Unlike existing approaches that rely on fixed cube decomposition strategies, the framework implements adaptive timeout mechanisms and dynamic work redistribution to handle heavy-tailed problem distributions. We demonstrate the framework's effectiveness on standard benchmarks and discuss future directions for search space partitioning heuristics.

Contents

1. Context

The Boolean Satisfiability Problem (**SAT**) represents the foundational challenge in automated reasoning, seeking to determine if there exists an assignment of truth values to variables that renders a given propositional formula true. These formulas are typically expressed in Conjunctive Normal Form (**CNF**), which consists of a conjunction of clauses, each representing a disjunction of literals. Consider the formula in CNF form $F = (x1 \vee x2) \wedge (\neg x1 \vee x3) \wedge (\neg x2 \vee \neg x3)$. A SAT solver aims to identify a satisfying assignment, such as $\{x1 = \top, x2 = \bot, x3 = \top\}$, which makes the formula evaluate to true.

This assessment of satisfiability is crucial across various domains, including hardware and software verification. For instance, verifying that a software program adheres to its specifications can be translated into a SAT problem, where the program's behavior is encoded as a propositional formula. Because of such wide-ranging applications, despite the NP-completeness of SAT, there has been significant progress in developing efficient solvers (notably CaDiCaL solver [1]) that can handle large and complex instances using sophisticated algorithmic techniques like Conflict-Driven Clause Learning (CDCL), chronological and non-chronological backtracking [2] to efficiently navigate and reduce the search space.

While SAT addresses the existence of a solution, Model Counting (**#SAT**) extends this problem by seeking to determine the total number of satisfying assignments within the 2^n possible configurations of a formula with n variables. This shift from decision to quantification elevates the problem to #P-complete [3], introducing a significantly higher level of computational complexity. However, #SAT enables a deeper analysis than classical SAT, providing insights into the solution space's structure and size. Particularly in the case of software verification, the number of satisfying assignments can be used to estimate the proportion of inputs for which the program behaves correctly, providing a probabilistic measure of correctness. Another prominent application of #SAT is in security analysis, where it can be used to evaluate the robustness of cryptographic protocols by not only determining if a vulnerability exists but also counting the number of ways an adversary can compromise a system and therefore assessing the risk level.

2. Motivation

Despite the advancements in current state-of-the-art #SAT solvers, many real-world instances remain computationally impractical for single-machine solving due to the exponential growth of the search space. Therefore, there is a pressing need for exploration of parrelization techniques in order to satisfy the demand of large-scale industrial benchmarks (e.g., circuit verification with millions of variables) or time-sensitive applications (e.g., probalistic inference in machine learning). Unfortunately, as more complex algorithms and heuristics are developed for model counting, it is not trivial to adapt them to a distributed setting since they often rely on intricate interactions between components (component caching, clause learning, etc).

However, recent work has shown that model counting do exhibits parallel characteristics when the search space is appropriately decomposed. The paper [4] demonstrates that by splitting the search space using cube decomposition, independent subproblems can be solved in parallel with minimal communication overhead. The cube-and-conquer paradigm [5] involves partitioning the search space into smaller "cubes" (partial assignments) and solving each cube independently, which can be effectively parallelized across multiple workers. While this approach has shown significant speedups for sequential solvers, it has not been fully explored in a distributed context, especially with adaptive strategies for handling heavy-tailed distributions of subproblem difficulty, where some subproblems may be solved very quickly, while others

may require significantly more time.

Accurately estimating the computational difficulty of these subproblems before solving them is challenging, and static partitioning strategies may lead to load imbalance and resource underutilization. Therefore, to address these problems, we propose the implementation of adaptive timeout mechanisms and dynamic work redistribution strategies. Furthermore, we intend to move beyond the constraints of traditional Message Passing Interface (**MPI**) by developing a flexible framework that supports heterogeneous deployment across mixed environments. By leveraging **gRPC** for inter-node communication, the solver can seamlessly integrate diverse resources, ranging from High-Performance Computing (HPC) clusters and cloud-based virtual machines to local workstations, into a unified distributed solving process. This modular architecture treats the underlying model counters as black boxes, allowing the framework to be easily adapted to any solver by simply wrapping its input and output interfaces within the distributed messaging layer.

3. Sequential #SAT solver background

3.1 Ganak2

For the primary evaluation of this work, Ganak2 [6] [7] is chosen and utilized as the core sequential backend. Ganak2 is a state-of-the-art model counter that builds upon the sharpSAT architecture, incorporating advanced component caching mechanisms where independent sub-formulas encountered during the search process are hashed and stored to avoid redundant computations. To further enhance efficiency, Ganak2 integrates the Arjun preprocessor to identify a minimal independent support of variables, effectively reducing the dimensionality of the search space from 2^n to 2^k, where $k \ll n$. The solver also leverages standard SAT-based heuristics, including Conflict-Driven Clause Learning (CDCL) and Variable State Independent Decaying Sum (VSIDS) branching, to prune the search space through conflict analysis. The latest version of Ganak2 as of 2026 has introduced multi-threading capabilities. This would actually act as a complementary parallelization layer to our distributed framework, allowing for efficient intra-node parallelism while our framework handles inter-node distribution of subproblems.

3.2 Alternative solvers

Alternative sequential engines with different flavors of heuristics and optimizations all should be compatible with our distributed framework as long as they can be invoked via command line and accept standard DIMACS CNF [8] input. Some notable names in current landscape of model counting include:

- **d4** : a knowledge compilation-based counter that compiles formulas into decision-DNNFs [9]
- **ApproxMC** [10] : an approximate model counter that provides probabilistic guarantees on the count. It is particularly useful for large instances where exact counting is infeasible or when an estimate is sufficient for the application at hand, offering a trade-off between accuracy and computational resources.
- **SharpSAT** [11] : A component-caching counter upon which Ganak2 is built, and its successor **SharpSAT-TD** [12] with the introduction of tree-decomposition techniques [13] to further optimize the search process.

4. My contribution

4.1 Work parition

Formally, a cube C is a conjunction of literals $l_1 \wedge l_2 \wedge ... \wedge l_k$ representing a partial assignment to a subset of variables in formula F. To partition the search space, a set of m cubes $C_1, C_2, ..., C_m$ is generated such that they cover all possible assignments and do not overlap. This requires that their disjunction is a tautology:

$$\bigvee_{i=1}^{m} C_i \equiv \top$$

and that for any $i \neq j$, the cubes are contradictory, $C_i \wedge C_j \equiv \bot$. Under these conditions, the total model count $\#(F)$ is the sum of the counts of the individual subproblems:

$$\#(F) = \sum_{i=1}^{m} \#(F \wedge C_i)$$

This additive property allows each worker in a distributed system to solve $F \wedge C_i$ independently. Since the search space is split into disjoint parts, the final result is obtained by a simple summation of the integer counts returned by each worker, requiring no further coordination between nodes during the solving phase.

4.2 General schema

4.3 Heuristics for work partition

5. Implementation and evaluation

5.1 Implementation

5.2 Evaluation

6. Conclusion and future work

Bibliography

[1] Armin Biere et al. "CaDiCaL 2.0". In: *Computer Aided Verification - 36th International Conference, CAV 2024, Montreal, QC, Canada, July 24-27, 2024, Proceedings, Part I*. Ed. by Arie Gurfinkel and Vijay Ganesh. Vol. 14681. Lecture Notes in Computer Science. Springer, 2024, pp. 133–152. DOI: 10.1007/978-3-031-65627-9_7.

[2] Robin Coutelier. "Chronological vs. non-chronological backtracking in satisfiability modulo theories". Unpublished master's thesis. MA thesis. Liège, Belgique: Université de Liège, 2023. URL: https://matheo.uliege.be/handle/2268.2/18041.

[3] Hans van Maaren Armin Biere Marijn Heule and Toby Walsch. "Handbook of Satisfiability". In: (2008). URL: https://www.cs.cornell.edu/~sabhar/chapters/ModelCounting-SAT-Handbook-prelim.pdf.

[4] Zhenghang Xu, Minghao Yin, and Jean Marie Lagniez. "An Embarrassingly Parallel Model Counter". In: *Proceedings of the 22nd International Conference on Principles of Knowledge Representation and Reasoning*. Oct. 2025, pp. 670–681. DOI: 10.24963/kr.2025/65. URL: https://doi.org/10.24963/kr.2025/65.

[5] Marijn Heule et al. "Cube and conquer: guiding CDCL SAT solvers by lookaheads". In: vol. 7261. Dec. 2011, pp. 50–65. ISBN: 978-3-642-34187-8. DOI: 10.1007/978-3-642-34188-5_8.

[6] Mate Soos and Kuldeep S. Meel. "Engineering an Efficient Probabilistic Exact Model Counter". In: *Proceedings of the International Conference on Computer Aided Verification (CAV)*. 2025.

[7] Shubham Sharma et al. "GANAK: A Scalable Probabilistic Exact Model Counter". In: *Proceedings of International Joint Conference on Artificial Intelligence (IJCAI)*. 2019.

[8] *SAT Competition 2009: Benchmark Submission Guidelines*. https://web.archive.org/web/20190325181937/https://www.satcompetition.org/2009/format-benchmarks2009.html. Accessed: 202X-XX-XX. 2009.

[9] Umut Oztok and Adnan Darwiche. "On Compiling CNF into Decision-DNNF". In: *Principles and Practice of Constraint Programming*. Ed. by Barry O'Sullivan. Cham: Springer International Publishing, 2014, pp. 42–57. ISBN: 978-3-319-10428-7.

[10] Yash Pote, Kuldeep S. Meel, and Jiong Yang. "Towards Real-Time Approximate Counting". In: *Proceedings of the AAAI Conference on Artificial Intelligence* 39.11 (Apr. 2025), pp. 11318–11326. DOI: 10.1609/aaai.v39i11.33231. URL: https://ojs.aaai.org/index.php/AAAI/article/view/33231.

[11] Marc Thurley. "sharpSAT: counting models with advanced component caching and implicit BCP". In: *Proceedings of the 9th International Conference on Theory and Applications of Satisfiability Testing*. SAT'06. Seattle, WA: Springer-Verlag, 2006, pp. 424–429. ISBN: 3540372067. DOI: 10.1007/11814948_38. URL: https://doi.org/10.1007/11814948_38.

[12] Tuukka Korhonen and Matti Järvisalo. *SharpSAT-TD in Model Counting Competitions 2021-2023*. 2023. arXiv: 2308.15819 [cs.AI]. URL: https://arxiv.org/abs/2308.15819.

[13] Tuukka Korhonen and Matti Järvisalo. "Integrating Tree Decompositions into Decision Heuristics of Propositional Model Counters". In: *27th International Conference on Principles and Practice of Constraint Programming (CP 2021)*. Ed. by Laurent D. Michel. Vol. 210. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021, 8:1–8:11. ISBN: 978-3-95977-211-2. DOI: 10.4230/LIPIcs.CP.2021.8. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.CP.2021.8.